LOAN DEFAULT RISK PREDICTION USING RANDOM FOREST

A Machine Learning Laboratory Project

Problem type: Supervised binary classification (class-imbalanced)

Algorithm: Random Forest Classifier (scikit-learn)

Dataset: Give Me Some Credit — Kaggle competition

Records: 150,000 labelled borrowers, 10 predictors

Deliverable: FastAPI backend with a React + TypeScript frontend

Model version: 1.0.0 (trained 2026-10-13, seed 42)

All results in this report were generated by executing the project code.

Abstract

This project implements an end-to-end machine learning system that estimates the probability of a borrower experiencing serious delinquency — 90 days past due or worse — within two years of an observation date. A Random Forest classifier was trained on the public Kaggle "Give Me Some Credit" dataset of 150,000 labelled borrower records with 10 numeric predictors. The positive class represents only 6.68% of records, making this a class-imbalanced binary classification problem in which accuracy is an actively misleading metric.

Five Random Forest configurations were compared on a held-out validation split using the area under the precision-recall curve as the selection criterion. The chosen configuration achieved a **ROC-AUC of 0.8654** and a **PR-AUC of 0.4014** on a test split of 30,000 records that was scored exactly once. The decision threshold was selected on validation data by maximising F1, giving a positive-class F1 of **0.4482** and recall of **0.4868** — against 0.2506 and 0.1566 at the conventional 0.50 cut-off. Five-fold cross-validation confirmed the stability of these estimates. The trained pipeline is served through a FastAPI REST API consumed by a four-page React frontend.

1. Introduction and Objective

1.1 Problem statement

Credit providers need to estimate the likelihood that an applicant will fall seriously behind on repayments. This project treats that need as a supervised learning problem and implements a complete, reproducible solution: data inspection, preprocessing, model training and evaluation, and deployment behind a web interface.

1.2 Type of machine learning problem

The task is **supervised binary classification**, for three reasons:

- **Supervised** — every training record carries a known outcome label, so the model learns from answers it is shown.
- **Classification, not regression** — the quantity predicted is a category (delinquent or not), not a continuous value. The model does emit a probability between 0 and 1, but that expresses confidence in a class rather than a magnitude.

- **Not clustering** — clustering is unsupervised and discovers groupings in unlabelled data. Here the two groups are defined in advance by the dataset.

A defining property of this particular problem is **class imbalance**: only 6.68% of borrowers are positive. This shapes every evaluation decision taken later in the report.

1.3 Scope

In scope: dataset research, data inspection and leakage checking, preprocessing without train-serve skew, Random Forest training and evaluation, a REST API, and a web interface. Deliberately out of scope: alternative algorithms, extensive hyperparameter search, databases, authentication and containerisation. The aim is to implement and understand one algorithm correctly rather than to maximise a leaderboard score.

2. Dataset

2.1 Source and provenance

The dataset is **Give Me Some Credit**, a Kaggle competition dataset available at https://www.kaggle.com/competitions/GiveMeSomeCredit/data. The data is credited to Credit Fusion and Kaggle and describes United States consumer credit behaviour from around 2011. Access requires a Kaggle account and acceptance of the competition rules; the raw files are therefore not redistributed with the project.

Only the labelled training file, cs-training.csv, is used. The competition also ships an unlabelled test file whose target column is entirely blank; no metric in this project is computed on it, because no honest metric could be. All reported figures come from splits of the labelled file.

2.2 Target variable

The target is **SeriousDlqin2yrs**, a binary indicator. A value of 1 means the borrower borrower experienced serious delinquency (90 days past due or worse) within two years of the observation date. A value of 0 means no such delinquency was recorded in that window.

This definition is narrow and the project states it consistently. The model does not predict loan default in general, nor does it indicate whether a borrower will ultimately repay.

2.3 Predictors

The 10 input features, all numeric, are listed in Table 1.

#	Feature	Description
1	RevolvingUtilizationOfUnsecuredLines	Card and personal line balances divided by total credit limits
2	age	Borrower age in years
3	NumberOfTime30-59DaysPastDueNotWorse	Times 30-59 days past due in the last two years
4	DebtRatio	Debt payments, alimony and living costs divided by gross monthly income
5	MonthlyIncome	Gross monthly income (19.82% missing)
6	NumberOfOpenCreditLinesAndLoans	Open loans and lines of credit
7	NumberOfTimes90DaysLate	Times 90 or more days past due
8	NumberRealEstateLoansOrLines	Mortgages and home equity lines
9	NumberOfTime60-89DaysPastDueNotWorse	Times 60-89 days past due in the last two years
10	NumberOfDependents	Dependents excluding the borrower (2.62% missing)

Table 1 — Input features, as verified against the competition data dictionary.

2.4 Class distribution

The data was split 60/20/20 with stratification, preserving the class ratio in each part. Table 2 confirms the balance is identical across splits.

Split	Records	Negative (0)	Positive (1)	Positive rate
Train	90,000	83,984	6,016	6.68%
Validation	30,000	27,995	2,005	6.68%
Test	30,000	27,995	2,005	6.68%

Table 2 — Stratified split sizes and class balance.

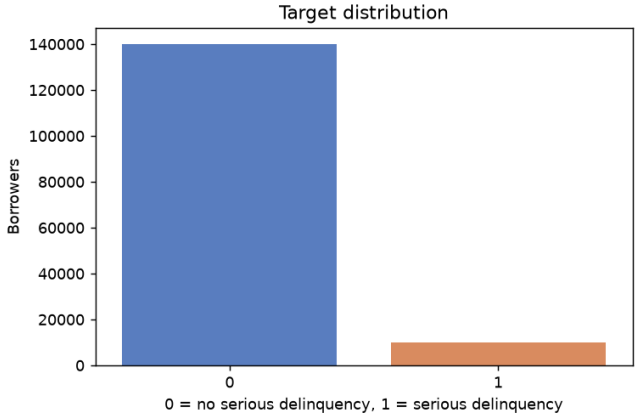


Figure 1 — Target distribution. The minority class is a small fraction of all records.

3. Exploratory Data Analysis

The raw file was inspected before any preprocessing logic was written. Key findings are summarised in Table 3.

Finding	Count	Decision taken
MonthlyIncome missing	29,731 (19.82%)	Median imputation plus indicator column
NumberOfDependents missing	3,924 (2.62%)	Median imputation plus indicator column
Delinquency counters coded 96 or 98	269 rows	Treated as placeholders; imputed with indicator
age equal to 0	1 row	Impossible; treated as missing
Duplicate predictor rows	646	Kept — plausibly distinct borrowers
Revolving utilisation above 1.0	3,321	Kept — over-limit balances are genuine
DebtRatio above 10	28,877	Kept — concentrated where income is unrecorded

Table 3 — Data-quality findings from ml/explore.py and the decisions they drove.

3.1 Target leakage check

No predictor correlates with the target above an absolute value of 0.13, and none is derived from the label. The delinquency counters describe behaviour in the two years *before* the observation date, whereas the target describes the two years *after* it, so they constitute legitimate history rather than leakage. Critically, the dataset contains no post-outcome fields such as recovery amounts or charge-off flags.

This check was a deciding factor in dataset selection. An alternative candidate, the Kaggle "Loan Default Prediction" dataset, was inspected and rejected because it carries fields such as recoveries, collection_recovery_fee and total_rec_late_fee, which are only known after a loan has already defaulted and would leak the answer directly into the model.

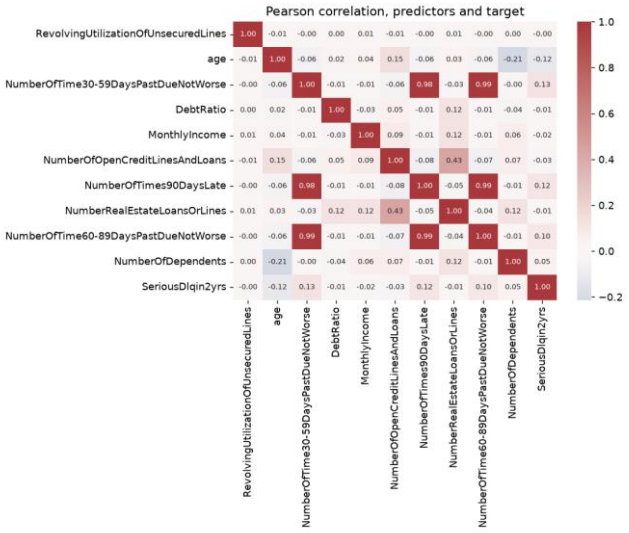


Figure 2 — Pearson correlation between predictors and the target.

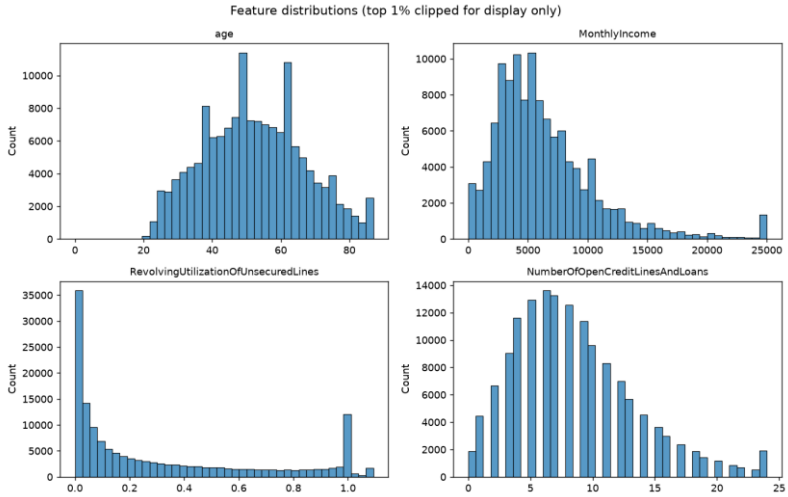


Figure 3 — Distributions of four representative features (top 1% clipped for display only).

4. Methodology

4.1 Preprocessing pipeline

Preprocessing and the classifier are wrapped in a single scikit-learn Pipeline, which is the object saved to disk. Keeping the transformations inside the pipeline is the project's main defence against train-serve skew: the API loads this object and calls it, so inference cannot apply a different transformation from the one fitted during training. There is no second copy of the preprocessing logic on the serving side.

The pipeline has three stages:

- **ImplausibleValueCleaner** — replaces documented placeholder and impossible values with missing markers: the codes 96 and 98 in the three delinquency counters, and any age below 18.
- **SimpleImputer(strategy="median", add_indicator=True)** — fills missing values with the training median and appends a binary column recording that the value was absent.
- **RandomForestClassifier** — the model itself.

All transformations are fitted on the training split only. A fixed random seed of 42 makes every result reproducible.

4.2 Handling of placeholder codes

The three past-due counters use 96 and 98 as codes rather than counts; a borrower cannot have been 30-59 days late 98 times in a 24-month window, and all 269 affected rows carry a code in all three counters simultaneously. The data dictionary does not document them.

These rows are **54.65% positive** against a base rate of 6.68%, so whatever the codes mean, they are strongly associated with the outcome. Imputing them to the median alone would tell the model those borrowers had a clean payment record. Asserting that 98 means "very many late payments" would instead claim a meaning the documentation does not support. The project therefore imputes the value but adds a missing-value indicator, allowing the model to learn that the figure was unavailable without anyone guessing why.

4.3 Why missing-value indicators were added

The imputer appends one binary column per feature that had missing values in training, giving the classifier **16 input columns** derived from the 10 input features. This was tested rather than assumed: an otherwise identical configuration without indicators was included in the comparison, and the indicator variant won on validation PR-AUC. The API request schema is unchanged, because the indicators are derived inside the pipeline.

4.4 Model selection

Five configurations were fitted on the training split and compared on the validation split. The selection metric is **average precision (PR-AUC)**, chosen because it reflects performance on the rare positive class, whereas accuracy is dominated by the majority class and ROC-AUC is comparatively optimistic under heavy imbalance.

Candidate	ROC-AUC	PR-AUC	Precision	Recall
baseline	0.8450	0.3481	0.5334	0.1870
regularised	0.8615	0.3820	0.5988	0.1556
balanced	0.8602	0.3776	0.2399	0.7197
balanced_subsample	0.8602	0.3780	0.2805	0.6504
regularised_with_indicators	**0.8630**	**0.3863**	**0.6290**	**0.1581**

Table 4 — Validation comparison of candidate configurations. The selected row is shown in bold.

The selected configuration is **regularised_with_indicators**: 300 trees with min_samples_leaf = 20, no class weighting, plus missing-value indicators. It was then refitted on train and validation combined (120,000 records) and scored once on the test split.

4.5 Finding on class weighting

Class weighting was investigated, as the imbalance warrants. Both class_weight="balanced" and "balanced_subsample" raised recall dramatically — from roughly 0.16 to 0.72 — while lowering precision by a comparable amount, yet left PR-AUC and ROC-AUC essentially unchanged. In other words, class weighting moved the model **along** the precision-recall curve rather than improving the curve itself.

Since a decision threshold achieves the same movement, but is visible in the saved metadata, adjustable without refitting, and chosen by a documented criterion, the unweighted model was retained and the operating point set explicitly by threshold selection.

4.6 Decision threshold selection

The conventional 0.50 cut-off is a poor fit at a 6.68% base rate: it makes the forest so conservative that it flags only 501 of 30,000 test borrowers and misses most genuine delinquencies.

The threshold is therefore **selected, not assumed**. It is chosen by maximising F1 on the **validation** split, scored by the candidate fitted on the training split only — a model that had never seen those records. The test split plays no part in the choice. The selected value is **0.2302**, and it is stored in the model metadata and read by the API at startup, so the threshold the service applies is always the one the reported metrics were measured at.

Validation F1 at this threshold was 0.4396, and test F1 came out at 0.4482. The close agreement is evidence that the threshold generalises rather than having been over-fitted to the validation split.

5. Results

5.1 Performance across all splits

Because the final model was refitted on train and validation combined, those two splits are in-sample; only the test column is a held-out estimate. All three are reported so the generalisation gap is visible rather than implied.

Metric	Train (in-sample)	Validation (in-sample)	Test (held out)
Accuracy	0.9264	0.9265	**0.9199**
Precision	0.4580	0.4565	**0.4153**
Recall	0.5472	0.5267	**0.4868**
F1-score	0.4986	0.4891	**0.4482**
ROC-AUC	0.9329	0.9333	**0.8654**
PR-AUC	0.5232	0.5105	**0.4014**

Table 5 — Metrics on every split at threshold 0.2302.

5.2 Confusion matrix

	Predicted 0	Predicted 1	Row total
Actual 0 (no delinquency)	26,621 (TN)	1,374 (FP)	27,995
Actual 1 (delinquency)	1,029 (FN)	976 (TP)	2,005

Table 6 — Test-set confusion matrix.

Read plainly: of 2,005 borrowers who genuinely became seriously delinquent, the model identified **976** and missed 1,029. Of 27,995 who did not, it incorrectly flagged 1,374.

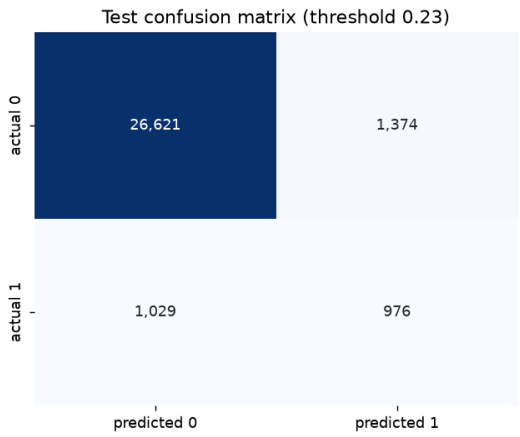


Figure 4 — Test confusion matrix.

5.3 Classification report

Class	Precision	Recall	F1-score	Support
0 — No serious delinquency	0.9628	0.9509	0.9568	27,995
1 — Serious delinquency	0.4153	0.4868	0.4482	2,005
Accuracy			**0.9199**	30,000
Macro average	0.6891	0.7189	**0.7025**	30,000
Weighted average	0.9262	0.9199	0.9228	30,000

Table 7 — Per-class classification report on the test split.

This table is the clearest demonstration of why a single headline number misleads on imbalanced data. The model is excellent on the majority class, with an F1 of 0.9568, and considerably weaker on the minority class at 0.4482. The weighted average of 0.9228 looks impressive but is dominated by the 27,995 negative records and conveys almost nothing about the class the project exists to detect.

Where a single figure is required, the **macro average F1 of 0.7025** or the **positive-class F1 of 0.4482** should be quoted, never the weighted average.

5.4 Why accuracy is not the headline metric

The model attains an accuracy of 0.9199. A trivial classifier that ignores all inputs and always predicts "no delinquency" would score **0.9332** on the same test set — a *higher* figure. Selecting a threshold that detects delinquencies deliberately reduced accuracy while tripling recall, which is the correct trade for this problem and the clearest possible demonstration that accuracy must not drive the decision.

The two metrics that carry information are ROC-AUC and PR-AUC. **ROC-AUC of 0.8654** means that, given one delinquent and one non-delinquent borrower drawn at random, the model assigns the delinquent borrower a higher score 86.5% of the time. **PR-AUC of 0.4014** should be read against a no-skill baseline equal to the positive rate, 0.0668 — approximately a sixfold improvement over chance on the rare class.

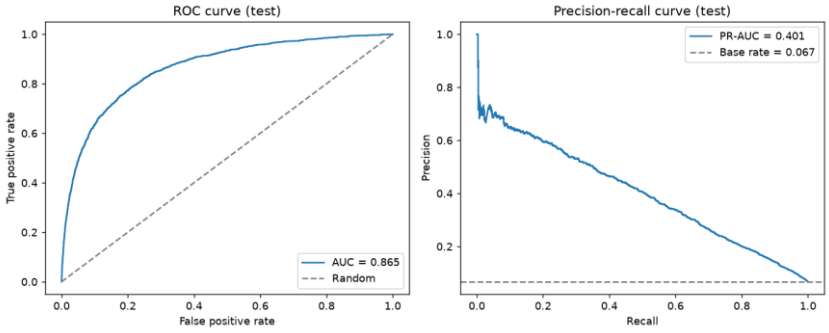


Figure 5 — ROC curve (left) and precision-recall curve (right) on the test split.

5.5 Threshold trade-off

The same trained model produces very different operating points depending on the cut-off applied. Table 8 shows the trade-off; "flagged" is the number of test borrowers marked positive.

Threshold	Accuracy	Precision	Recall	F1	Flagged
0.1000	0.8469	0.2610	0.7047	0.3810	5,413
0.2000	0.9131	0.3884	0.5222	0.4454	2,696
0.2302	0.9199	0.4153	0.4868	0.4482	2,350
0.3000	0.9292	0.4650	0.3970	0.4283	1,712
0.5000	0.9374	0.6267	0.1566	0.2506	501

Table 8 — Precision-recall trade-off across decision thresholds.

Lowering the threshold to 0.10 raises recall above 0.70 but reduces precision to roughly 0.26, meaning three of every four flagged borrowers would be false alarms. Raising it to 0.50 yields precision above 0.62 but detects only about one delinquency in six. Accuracy increases monotonically with the threshold purely because the model predicts the majority class more often, which is precisely why it is a poor guide here.

5.6 Generalisation gap

Comparing in-sample with held-out performance quantifies how much of the apparent skill is memorisation.

Metric	Train	Test	Gap (train − test)
ACCURACY	0.9264	0.9199	+0.0065
PRECISION	0.4580	0.4153	+0.0427
RECALL	0.5472	0.4868	+0.0604
F1	0.4986	0.4482	+0.0504
ROC-AUC	0.9329	0.8654	+0.0675
PR-AUC	0.5232	0.4014	+0.1218

Table 9 — Generalisation gap between the fitted and held-out splits.

The model still overfits: ROC-AUC falls by 0.0675 and PR-AUC by 0.1218 from in-sample to held out. Regularisation through min_samples_leaf reduced this materially relative to a depth-unlimited forest, and the indicator columns narrowed it slightly further, but 300 trees grown on 120,000 records retain substantial capacity to memorise. Notably, accuracy barely moves while PR-AUC collapses — further evidence that accuracy is insensitive to what matters here.

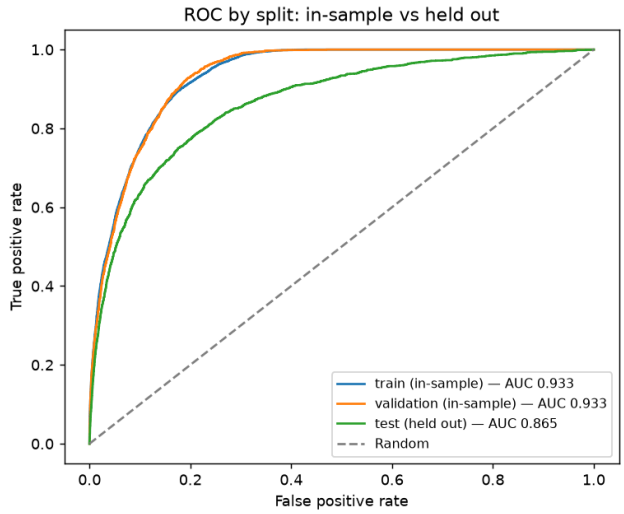


Figure 6 — ROC curves by split. The separation between the in-sample and held-out curves is the generalisation gap.

5.7 Cross-validation

A single test score derives from one particular partition. Stratified five-fold cross-validation was run on the train and validation data combined, with the test split excluded throughout and the same decision threshold applied, to establish whether that score is stable.

Metric	CV mean	Std. dev.	Min	Max	Test
ACCURACY	0.9189	0.0023	0.9166	0.9232	0.9199

Metric	CV mean	Std. dev.	Min	Max	Test
PRECISION	0.4094	0.0141	0.3938	0.4345	0.4153
RECALL	0.4792	0.0120	0.4589	0.4963	0.4868
F1	0.4415	0.0130	0.4238	0.4633	0.4482
ROC-AUC	0.8632	0.0053	0.8591	0.8733	0.8654
PR-AUC	0.3978	0.0148	0.3841	0.4231	0.4014

Table 10 — Five-fold cross-validation summary compared with the held-out test result.

Every test metric falls within one standard deviation of its cross-validated mean, and the spread across folds is small. The held-out estimate is therefore not an artefact of one fortunate split; the performance is a property of the data and configuration. A minor caveat is that each fold trains on fewer records than the final model, making cross-validation very slightly pessimistic.

5.8 Feature importance

Rank	Feature	Importance
1	NumberOfTimes90DaysLate	0.2672
2	RevolvingUtilizationOfUnsecuredLines	0.2076
3	NumberOfTime60-89DaysPastDueNotWorse	0.1361
4	NumberOfTime30-59DaysPastDueNotWorse	0.1096
5	DebtRatio	0.0688
6	age	0.0581
7	MonthlyIncome	0.0538
8	NumberOfOpenCreditLinesAndLoans	0.0448
9	NumberRealEstateLoansOrLines	0.0172
10	NumberOfDependents	0.0147
11	missingindicator_NumberOfTime30-59DaysPastDueNotWorse	0.0064
12	missingindicator_NumberOfTime60-89DaysPastDueNotWorse	0.0062
13	missingindicator_NumberOfTimes90DaysLate	0.0062
14	missingindicator_MonthlyIncome	0.0028
15	missingindicator_NumberOfDependents	0.0004
16	missingindicator_age	0.0000

Table 11 — Mean impurity-decrease importance for all model input columns.

Payment history dominates: the three past-due counters together account for the largest share of the total, followed by revolving credit utilisation. This is intuitive, but it must not be over-interpreted.

These values are not causal statements. Impurity-based importance records what the trees chose to split on, which is not the same as what causes delinquency. It is biased toward high-cardinality features, since continuous ratios offer far more candidate split points than counters that are zero for most borrowers. It is computed on training data. Correlated predictors divide the credit between them, so each past-due counter appears less important than past-due behaviour as a whole truly is. Finally, rare binary flags are structurally understated: the indicator columns rank last despite having won the model-selection comparison, because a flag set on a few hundred of 150,000 records cannot accumulate much total impurity reduction however decisive it is on those records.

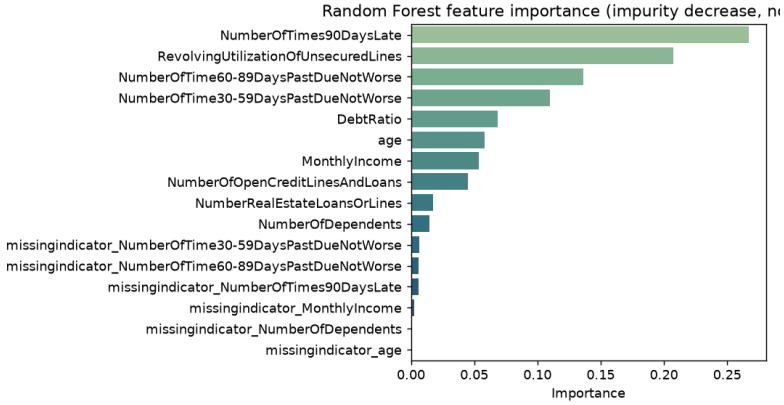


Figure 7 — Feature importance by mean impurity decrease.

6. System Implementation

6.1 Architecture

The trained pipeline is persisted with joblib and served by a FastAPI application that loads it once at startup, never per request. A React frontend consumes the API over HTTP.

Layer	Technology
Model	scikit-learn RandomForestClassifier, persisted with joblib
Data processing	pandas, NumPy; matplotlib and seaborn for figures
API	FastAPI, Pydantic v2, Uvicorn
Frontend	React 19, Vite, TypeScript, Tailwind CSS, React Router
Testing	pytest (61 tests)

Table 12 — Technology stack.

6.2 API endpoints

Method	Endpoint	Purpose
GET	/health	API status, model availability and version
GET	/model-info	Model, dataset, target, features, metrics, risk bands
GET	/demo-examples	Real held-out records for the demonstration button
POST	/predict	Score one borrower record
GET	/docs	Interactive Swagger documentation

Table 13 — REST API surface.

Requests are validated with Pydantic against the exact feature schema used in training. Invalid, missing, non-numeric, non-finite or unknown fields produce a 422 response identifying the offending field; a missing model artifact produces a 503. The positive-class probability is located by looking up label 1 in the model's classes_ attribute rather than assuming a column index, so a change in class ordering cannot silently invert predictions.

6.3 Frontend

The interface has four pages: a Dashboard summarising the project and live metrics, a Prediction form, a Results page, and a Model Information page. All metrics displayed are fetched from the API at runtime; none are hardcoded. Two interactive charts were added, each chosen because a table cannot show what it shows: a score-separation chart, drawing the predicted-score distribution of each actual outcome group, which explains why recall is capped rather than merely stating it; and a threshold curve, whose hover converts rates into counts of borrowers caught and wrongly flagged. The same separation chart appears on the results page with a marker for the borrower just scored. Charts that would only repeat a number, such as a ROC curve or confusion-matrix heatmap, were deliberately omitted. The form exposes only the ten features the model was actually trained on, with plain-language labels, grouped inputs and validation bounds taken from the data dictionary.

6.4 Testing

The project includes **61 automated tests** covering the dataset contract (verifying the target can never enter the feature list), placeholder-code and implausible-value cleaning, imputation behaviour, split determinism and disjointness, metric arithmetic against hand-checked examples, correct probability-column lookup under a reversed class ordering, artifact loading independent of the training code, and every API endpoint including its error paths.

6.5 Deployment

The application is hosted on Vercel as two projects. The frontend is available at https://delinquency-risk.vercel.app and the backend, with interactive Swagger documentation, at https://loan-default-api.vercel.app/docs. The 70 MB model file is bundled with the backend; the deployed bundle is about 315 MB, within the Vercel 500 MB limit for Python functions, which is possible because the runtime requirements were separated from the training and testing tools.

The browser never calls the backend directly. The frontend rewrites requests to /api onto the backend, so every request is same-origin and cross-origin resource sharing does not apply; the backend additionally allows only the frontend domains. The interface was verified at widths from 320 to 1280 pixels: charts are drawn at their real pixel width so labels remain legible on a phone, and tapping a chart opens the same tooltip as hovering. After a period of inactivity the first request takes roughly eight to nine seconds while the model loads.

7. Discussion and Limitations

7.1 Interpretation of the result

A ROC-AUC of 0.8654 indicates that the model extracts genuine, substantial signal from the data. The comparatively modest positive-class F1 of 0.4482 is not a defect of the implementation but a property of the problem: predicting individual human financial behaviour two years into the future from ten numeric attributes has an inherent ceiling. The original competition was itself scored on AUC rather than F1 for this reason.

7.2 Dataset limitations

- **Age of the data.** The records describe United States consumer credit around 2011. Lending standards, products and macroeconomic conditions have changed substantially since.
- **Unknown sampling frame.** The competition does not document how these borrowers were selected, so there is no basis for establishing which population the sample represents.
- **Fairness cannot be assessed.** The dataset contains no race, sex or geographic attributes, so no disparate-impact test is possible. Their absence is not protection: age is present and used, and features such as income and credit utilisation can act as

proxies. The absence of measured bias is the absence of measurement, not evidence of fairness.

7.3 Methodological limitations

- **Residual overfitting**, quantified in Section 5.6 and not eliminated.
- **No out-of-time validation.** The dataset has no date column, so the standard credit-model validation — testing on a later period — is impossible. A random split from a single snapshot is the easiest possible test.
- **Probabilities are not calibrated.** The output is the mean of the per-tree class probabilities, each tree contributing the class proportion of the leaf the borrower falls into. Random Forest probabilities are typically poorly calibrated, so a predicted 0.30 should not be read as thirty occurrences in a hundred. Calibration was not evaluated, and the interface describes the output as an estimated probability throughout.
- **The threshold optimises F1**, which weights a false positive and a false negative equally. Real lending economics do not: a missed delinquency usually costs far more than a declined sound borrower. Setting the threshold properly would require loss-given-default and margin figures the dataset does not contain.
- **Imputation blurs real signal.** For roughly one record in five the model has no actual income figure, only the training median and an indicator that it was absent.

7.4 Why this is an educational prototype

A system used for real lending decisions would additionally require current and representative data, out-of-time validation, fairness testing, calibrated and monitored probabilities, adverse-action reasoning for declines, regulatory compliance, model governance and audit trails, drift monitoring, human review of automated decisions, and authentication and logging. None of these are implemented. Beyond the technical gaps, credit decisions materially affect people and are regulated accordingly; this project must not be used to approve, decline or price real credit.

8. Conclusion

This project delivers a complete, reproducible, supervised binary classification system for predicting serious borrower delinquency. A Random Forest classifier trained on 150,000 labelled records achieved a **ROC-AUC of 0.8654** and a **PR-AUC of 0.4014** on a held-out

test split scored exactly once, with a positive-class F1 of **0.4482** and a macro-average F1 of **0.7025** at a validation-selected decision threshold of 0.2302. Five-fold cross-validation confirmed these estimates are stable rather than artefacts of one partition.

Beyond the numbers, the project demonstrates several methodological points that matter more than any single score: that accuracy is actively misleading under class imbalance, that class weighting and threshold adjustment move a model along the same precision-recall curve rather than improving it, that missingness can itself be informative and should be tested rather than assumed, and that the decision threshold is a modelling choice to be selected on validation data rather than inherited as a default. Preprocessing is held inside the persisted pipeline so that training and inference cannot diverge, and every reported figure is regenerated from the artifacts on each run rather than transcribed.

8.1 Future work

- Evaluate probability calibration using CalibratedClassifierCV, a reliability diagram and the Brier score.
- Compute permutation importance on the test split, which is less biased than impurity-based importance.
- Select the decision threshold against an explicit cost matrix rather than by maximising F1.
- Add per-prediction explanations, for example SHAP values, to the results page.
- Extend automated testing to the frontend components.

References

[1] Give Me Some Credit, Kaggle competition. Data credited to Credit Fusion and Kaggle. https://www.kaggle.com/competitions/GiveMeSomeCredit/data

[2] Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2825-2830.

[3] Breiman, L. (2001). Random Forests. Machine Learning, 45(1), 5-32.

[4] Saito, T. and Rehmsmeier, M. (2015). The Precision-Recall Plot Is More Informative than the ROC Plot When Evaluating Binary Classifiers on Imbalanced Datasets. PLOS ONE, 10(3).

Project documentation accompanying this report: README.md (overview and results), SETUP.md (installation and run guide), docs/dataset-research.md, docs/methodology.md, docs/model-evaluation.md and docs/limitations.md. Model artifact version 1.0.0, trained 2026-10-13 with random seed 42.